

Fiche 307 — Lire des données : `read.table()`, `scan()`, jeux intégrés

Matière	Code · Langage R
Cours source	R Core Team, <i>An Introduction to R</i> 4.6.1 — chapitre 7 « Reading data from files » (§7.1 <code>read.table()</code> , §7.2 <code>scan()</code> , §7.3 jeux de données intégrés, §7.3.1 données d'autres paquets, §7.4 édition)
Sources d'appoint	R Core Team, <i>R Data Import/Export</i> (renvoi explicite du §7) ; <i>An Introduction to R</i> §3.2 (ajustement automatique des longueurs)
Difficulté	Intermédiaire — court, mais c'est la porte d'entrée de toute analyse réelle
Temps d'étude estimé	45 min
Prérequis	Fiches 301, 303, 306 (vecteurs, modes, listes et data frames)
Concepts clés	forme attendue du fichier, étiquettes de ligne implicites , <code>read.table()</code> , <code>header = TRUE</code> , lecture par défaut des types, <code>scan()</code> , liste fictive qui fixe les modes, <code>scan()</code> à valeur unique, <code>matrix(scan(...), ncol =, byrow = TRUE)</code> , <code>data()</code> , <code>data(package =)</code> , <code>edit()</code> , <code>fix()</code>
À retenir en priorité	La règle « un item de moins sur la première ligne » · <code>header = TRUE</code> · la liste fictive de <code>scan()</code> · <code>fix(x) = x <- edit(x)</code> .

Vue d'ensemble

LA PHILOSOPHIE « R input facilities are SIMPLE and their requirements are FAIRLY STRICT and even RATHER INFLEXIBLE »
-> on nettoie le fichier AVANT, avec d'autres outils

DEUX PORTES `read.table()` lit un DATA FRAME ENTIER <- la voie normale
`scan()` plus primitif, lit des VECTEURS

LA REGLE DU FICHIER
1re ligne : UN NOM PAR VARIABLE

lignes suivantes : ETIQUETTE DE LIGNE puis les valeurs
« If the file has ONE FEWER ITEM in its first line than in its
second, this arrangement is PRESUMED to be in force. »

SANS ETIQUETTES `read.table("f.data", header = TRUE)`

TYPES PAR DEFAUT numérique -> variable numerique
 non numerique -> variable CARACTERE

`scan()` `inp <- scan("input.dat", list("", 0, 0))`
le 2e argument est une LISTE FICTIVE qui FIXE LES MODES
nommee -> `inp$id`, `inp$x`, `inp$y`
valeur unique -> UN SEUL vecteur, tout du meme mode

JEUX INTEGRES ~100 dans le paquet datasets ; `data()` les liste
 `data(package = "rpart")` `data(Puromycin, package = "datasets")`

EDITER `xnew <- edit(xold)` `fix(xold) == xold <- edit(xold)`
 `edit(data.frame())` pour saisir de zero

Le problème posé. « *Large data objects will usually be read as values from external files rather than entered during an R session at the keyboard.* » (§7) Jusqu'ici, toutes les données ont été tapées à la main. Aucune analyse réelle ne fonctionne ainsi.

La mise en garde qui ouvre le chapitre, et qu'il faut prendre au sérieux. « *R input facilities are simple and their requirements are fairly strict and even rather inflexible. There is a clear presumption by the designers of R that you will be able to modify your input files using other tools, such as file editors or Perl, to fit in with the requirements of R. Generally this is very simple.* »

R **n'essaie pas de deviner** un fichier mal formé. C'est un choix de conception, pas une lacune : le nettoyage se fait **avant**, avec l'outil approprié — un éditeur, `sed`, `awk` (note 1 : « *Under UNIX, the utilities `sed` or `awk` can be used* »), Perl.

Le renvoi (§7). « *For more details on importing data into R and also exporting data, see R Data Import/Export.* » — un manuel officiel distinct, non couvert par ce chapitre.

● Concept 1 — La forme attendue du fichier

Règle (§7.1). « *To read an entire data frame directly, **the external file will normally have a special form** :*

- « ***The first line of the file should have a name for each variable** in the data frame. »*
- « ***Each additional line of the file has as its first item a row label** and the values for each variable. »*

La règle de détection, à connaître par cœur (§7.1). « *If the file has one fewer item in its first line than in its second, this arrangement is presumed to be in force. »*

R compte les items. Si la ligne d'en-tête en a **un de moins** que la première ligne de données, R en conclut que la colonne supplémentaire des lignes de données est **une colonne d'étiquettes**. Aucune option n'a été donnée : **la structure du fichier a suffi**.

La forme avec étiquettes de ligne (§7.1) :

	Price	Floor	Area	Rooms	Age	Cent.heat
01	52.00	111.0	830	5	6.2	no
02	54.75	128.0	710	5	7.5	no
03	57.50	101.0	1000	5	4.2	no
04	57.50	131.0	690	6	8.8	no
05	59.75	93.0	900	5	1.9	yes

Six noms sur la première ligne, sept items sur les suivantes : **la règle s'applique**, et 01 , 02 ... deviennent les **étiquettes de ligne** (`row.names` , fiche 306).

```
HousePrice <- read.table("houses.data")
```

Règle — les types lus par défaut (§7.1). « *By default numeric items (except row labels) are read as numeric variables and non-numeric variables, such as `Cent.heat` in the example, as character variables. This can be changed if necessary.* »

« **Except row labels** ». Les étiquettes `01` , `02` **ne deviennent pas** une variable numérique : ce sont des `row.names` , donc des chaînes. C'est ce qui préserve le zéro initial.

La forme sans étiquettes (§7.1) :

Price	Floor	Area	Rooms	Age	Cent.heat
52.00	111.0	830	5	6.2	no
54.75	128.0	710	5	7.5	no
57.50	101.0	1000	5	4.2	no
57.50	131.0	690	6	8.8	no
59.75	93.0	900	5	1.9	yes

```
HousePrice <- read.table("houses.data", header = TRUE)
```

DÉFINITION (§7.1).

« where **the** `header=TRUE` **option specifies that the first line is a line of headings, and hence, by implication from the form of the file, that no explicit row labels are given** ».

Les deux formes sont symétriques, et c'est le décompte des items qui les sépare :

Fichier	Items ligne 1	Items ligne 2	Appel
avec étiquettes	<i>p</i>	<i>p + 1</i>	<code>read.table("f")</code> — détection automatique
sans étiquettes	<i>p</i>	<i>p</i>	<code>read.table("f", header = TRUE)</code>

Le cas mortel est celui où l'on oublie `header = TRUE` **sur un fichier sans étiquettes.** Les items s'équilibrent, R ne présume rien, et **la ligne de noms devient la première ligne de données.** Toutes les colonnes passent alors en **caractère** (puisqu'elles contiennent un nom), et l'analyse échoue plus loin sur un message qui ne parle pas de lecture.

● Concept 2 — `scan()` : la voie primitive

Cadrage (§7). « *There is also a more primitive input function, `scan()`, that can be called directly.* »

MÉTHODE (§7.2).

« *Suppose the data vectors are **of equal length and are to be read in parallel**. Further suppose that there are **three vectors, the first of mode character and the remaining two of mode numeric**, and the file is `input.dat`. **The first step is to use `scan()` to read in the three vectors as a list*** » :

```
inp <- scan("input.dat", list("", 0, 0))
```

DÉFINITION — LA LISTE FICTIVE (§7.2).

« *The second argument is a dummy list structure that establishes the mode of the three vectors to be read. The result, held in `inp`, is a list whose components are the three vectors read in.* »

C'est l'idée centrale du paragraphe. On ne dit pas à `scan()` *combien* de lignes lire, ni *quel* séparateur : on lui donne **un exemplaire vide de ce qu'on attend.** `""` annonce du caractère, `0` du numérique. La forme de l'argument **est** la spécification.

Récupérer les vecteurs (§7.2) :

```
label <- inp[[1]]; x <- inp[[2]]; y <- inp[[3]]
```

« **More conveniently, the dummy list can have named components**, in which case the names can be used to access the vectors read in » :

```
inp <- scan("input.dat", list(id = "", x = 0, y = 0))
label <- inp$id; x <- inp$x; y <- inp$y
```

« or **the list may be attached at position 2 of the search path** » — l'usage direct de `attach()` sur une liste quelconque (§6.3.4, fiche 306).

Règle — le cas du vecteur unique (§7.2). « *If the second argument is a single value and not a list, a single vector is read in, all components of which must be of the same mode as the dummy value.* »

```
X <- matrix(scan("light.dat", 0), ncol = 5, byrow = TRUE)
```

Trois choses en une ligne : `scan("light.dat", 0)` lit **un long vecteur numérique** ; `matrix(..., ncol = 5)` lui pose un `dim` (fiche 305) ; `byrow = TRUE` dit que le fichier est écrit **ligne par ligne**, alors que R remplit **en colonne-majeur**. Sans lui, la matrice serait **transposée et mélangée**.

« There are **more elaborate input facilities** available and these are detailed in the manuals. »

Énoncé. Trois fichiers à lire. Lequel des deux outils pour chacun, et avec quels arguments ?

1. `houses.data` — en-tête de 6 noms, 7 items par ligne de données.
2. `input.dat` — trois colonnes sans en-tête : un identifiant textuel, deux nombres.
3. `light.dat` — une longue suite de nombres, à lire comme une matrice à 5 colonnes.

Étape 1 — cas 1, compter les items. En-tête : 6. Données : 7. **Un de moins** sur la première ligne : « *this arrangement is **presumed** to be in force* ». La colonne supplémentaire est donc une colonne d'étiquettes.

```
HousePrice <- read.table("houses.data")
```

Aucun argument nécessaire — c'est tout l'intérêt de la règle de détection.

Étape 2 — vérifier ce qu'on a obtenu. `Cent.heat` contient `no / yes` : « **non-numeric variables ... as character variables** ». Pour en faire un facteur (fiche 304), il faudra une conversion explicite — le

cours dit seulement « *this can be changed if necessary* ».

Étape 3 — cas 2 : pas d'en-tête du tout. `read.table()` **suppose** un en-tête ou des étiquettes ; ici il n'y a ni l'un ni l'autre. C'est le terrain de `scan()` , et il faut lui **déclarer les modes** :

```
inp <- scan("input.dat", list(id = "", x = 0, y = 0))
```

"" pour le caractère, 0 pour les deux numériques. Nommer la liste fictive rend l'accès lisible : `inp$id` , `inp$x` , `inp$y` .

Étape 4 — pourquoi ne pas nommer par `[[ ]]` . On **peut** : `inp[[1]]` , `inp[[2]]` , `inp[[3]]` (\$7.2). Mais les noms « *can be used to access the vectors read in* » — et le cours qualifie cette voie de « **more conveniently** ». C'est aussi plus robuste : ajouter une colonne ne décale pas les indices.

Étape 5 — cas 3 : un seul mode, une forme matricielle. Le deuxième argument devient **une valeur unique** : « *a single vector is read in, all components of which **must be of the same mode as the dummy value*** ».

```
X <- matrix(scan("light.dat", 0), ncol = 5, byrow = TRUE)
```

Étape 6 — pourquoi `byrow = TRUE` est obligatoire ici. `scan()` rend les nombres **dans l'ordre du fichier**, c'est-à-dire ligne par ligne. `matrix()` remplit **en colonne-majeur** (fiche 305). Sans `byrow = TRUE` , la première colonne de `X` recevrait les cinq premiers nombres **du fichier**, qui en sont la première **ligne**. Le résultat aurait la bonne dimension et serait **entièrement faux**.

Étape 7 — la règle de choix. **Un data frame** → `read.table()` . **Des vecteurs parallèles de modes différents** → `scan()` avec une liste fictive nommée. **Un bloc homogène** → `scan()` avec une valeur unique, plus une mise en forme. Et dans le doute, le cours a une préférence : « ***If variables are to be held mainly in data frames, as we strongly suggest they should be, an entire data frame can be read directly with read.table().*** »

● Concept 3 — Les jeux de données intégrés

Règle (§7.3). *« **Around 100 datasets are supplied with R (in package `datasets`), and others are available in packages (including the recommended packages supplied with R).** To see the list of datasets currently available use `data()` . »*

« **All the datasets supplied with R are available directly by name. However, many packages still use the obsolete convention in which `data` was also used to load datasets into R** » — comme `data(infert)` , « and this can still be used with the standard packages ».

« In most cases this will **load an R object of the same name**. However, **in a few cases it loads several objects**, so **see the on-line help for the object to see what to expect**. »

Ce qu'il faut en retenir. `data(infert)` n'est plus nécessaire — les jeux du paquet `datasets` **sont directement disponibles par leur nom**. La forme `data(...)` reste utile pour les paquets qui n'ont pas migré, et **elle peut charger plusieurs objets d'un coup** : l'aide de l'objet dit lesquels.

MÉTHODE (§7.3.1).

« To access data from a particular package, **use the `package` argument** » :

```
data(package = "rpart")           # LISTER les jeux de rpart
data(Puromycin, package = "datasets") # CHARGER un jeu précis
```

« **If a package has been attached by `library`, its datasets are automatically included in the search.** » — cohérent avec le chemin de recherche de la fiche 306 : `library()` place le paquet **sur le chemin**, et ses données avec.

« **User-contributed packages can be a rich source of datasets.** »

● Concept 4 — Éditer

DÉFINITION (§7.4).

« When invoked on **a data frame or matrix**, `edit` brings up a separate spreadsheet-like environment for editing. This is useful for making small changes once a data set has been read. »

```
xnew <- edit(xold)           # edite xold, RANGE LE RESULTAT DANS xnew
fix(xold)                   # « equivalent to xold <- edit(xold) »
xnew <- edit(data.frame())  # saisir de NOUVELLES donnees
```

La subtilité de `edit()` (§7.4). « The command `xnew <- edit(xold)` will allow you to edit your data set `xold`, and on completion the changed object is assigned to `xnew` ». `xold` n'est pas modifié — c'est la sémantique de copie de R (fiche 302), une fois de plus.

« If you want to alter the original dataset `xold`, the simplest way is to use `fix(xold)`, which is equivalent to `xold <- edit(xold)`. »

Et l'usage de saisie à froid : « Use `xnew <- edit(data.frame())` to enter new data via the spreadsheet interface. » — `data.frame()` sans argument crée un data frame **vide**, que l'éditeur remplit.

Ce chemin est réservé aux petites retouches. Le cours le dit : « useful for making small changes once a data set has been read ». Une modification faite à la souris **ne laisse aucune trace reproductible** — elle ne figure ni dans un script, ni dans `.Rhistory` de façon exploitable. Tout ce qui doit être refait doit être **écrit**.

Comment reconnaître le type d'exercice

Le signal dans l'énoncé	Ce qu'il faut mobiliser
« lire un tableau de données »	<code>read.table()</code> — la voie normale
« la première ligne contient les noms »	compter les items ; sinon <code>header = TRUE</code>
« il y a une colonne d'identifiants en tête de ligne »	étiquettes de ligne — détection automatique
« mes colonnes sont toutes du texte »	<code>header = TRUE</code> oublié : la ligne de noms est passée en données
« ma variable <code>oui/non</code> n'est pas un facteur »	« <i>non-numeric ... as character variables</i> » — à convertir
« des vecteurs parallèles de modes différents »	<code>scan()</code> avec une liste fictive
« comment <code>scan</code> connaît-il les types ? »	par la liste fictive — <code>"</code> caractère, <code>0</code> numérique
« je veux accéder aux vecteurs par leur nom »	nommer la liste fictive
« un bloc de nombres à mettre en matrice »	<code>matrix(scan(f, 0), ncol = k, **byrow = TRUE**)</code>
« ma matrice est mélangée »	<code>byrow = TRUE</code> manquant — R remplit en colonne
« quels jeux de données ai-je ? »	<code>data()</code>
« un jeu d'un paquet précis »	<code>data(nom, package = "...")</code>
« corriger deux valeurs à la main »	<code>fix(x)</code> — et non <code>edit(x)</code> seul
« saisir un petit jeu de zéro »	<code>edit(data.frame())</code>
« mon fichier est mal formé »	le nettoyer avant — R est « <i>rather inflexible</i> »

Comment résoudre ce type d'exercice

Protocole « lire un fichier de données » — 5 étapes.

1. **Regarder le fichier** — `file.show()` ou un éditeur. Compter les items de la première et de la deuxième ligne.

2. **Choisir la forme** : un de moins sur la première → `read.table(f)` ; autant → `read.table(f, header = TRUE)` .
3. **Vérifier immédiatement** ce qui a été lu : `str()` , `names()` , `nrow()` , et surtout **le mode de chaque colonne**.
4. **Convertir ce qui doit l'être** — les variables catégorielles en facteurs (fiche 304), avec l'ordre de niveaux voulu.
5. Si le fichier résiste, **le corriger hors de R** plutôt que de chercher l'argument magique.

Protocole « `scan()` proprement » — 4 étapes.

1. Déterminer **combien de vecteurs** et **de quels modes**.
2. Écrire **la liste fictive nommée** : `list(id = "", x = 0, y = 0)` .
3. Récupérer par `inp$nom` — ou **attacher la liste** en position 2.
4. Si un seul mode : deuxième argument **scalaire**, puis mise en forme (`matrix` , `dim`) **sans oublier `byrow`** .

● Common mistakes

Erreur	Correction
Oublier <code>header = TRUE</code> sur un fichier sans étiquettes	la ligne de noms devient une ligne de données
Ajouter <code>header = TRUE</code> sur un fichier avec étiquettes	la détection automatique suffisait ; on décale tout
Croire que R devine n'importe quel format	« <i>fairly strict and even rather inflexible</i> »
Attendre un facteur d'une colonne <code>oui/non</code>	elle est lue en caractère
Croire les étiquettes de ligne lues comme une variable	« <i>except row labels</i> » — ce sont des <code>row.names</code>
Utiliser <code>read.table()</code> sur un fichier sans en-tête ni étiquettes	c'est le terrain de <code>scan()</code>
Croire que <code>scan()</code> devine les modes	c'est la liste fictive qui les fixe
Mélanger les modes dans un <code>scan()</code> à valeur unique	« <i>all components ... must be of the same mode</i> »
Oublier <code>byrow = TRUE</code> après un <code>scan()</code>	R remplit en colonne-majeur → matrice fausse
Croire <code>data(x)</code> nécessaire	les jeux de <code>datasets</code> sont directement disponibles
Croire que <code>data()</code> charge toujours un seul objet	« <i>in a few cases it loads several objects</i> »
Croire que <code>edit(x)</code> modifie <code>x</code>	il assigne le résultat — utiliser <code>fix(x)</code>
Corriger des données à la souris dans un travail sérieux	aucune trace reproductible

Ultimate Review

La philosophie. « *R input facilities are **simple** and their requirements are **fairly strict and even rather inflexible**. There is a clear presumption ... that **you will be able to modify your input files using other tools*** » — éditeur, `sed`, `awk`, Perl. Renvoi au manuel **R Data Import/Export**.

Deux outils. `read.table()` lit **un data frame entier** — « *if variables are to be held mainly in data frames, as we strongly suggest they should be* ». `scan()` est « *a more primitive input function* ».

La forme du fichier. (1) « *The first line of the file **should have a name for each variable*** » ; (2) « *Each additional line has **as its first item a row label** and the values* ». **La règle de détection** : « *If the file has*

one fewer item in its first line than in its second, this arrangement is presumed to be in force. »

Sans étiquettes de ligne : `header = TRUE`, qui « specifies that the first line is a line of headings, and hence, by implication from the form of the file, that no explicit row labels are given ».

Types par défaut. « *numeric items (except row labels) are read as numeric variables and non-numeric variables ... as character variables. This can be changed if necessary.* »

`scan()` . Deuxième argument = **une liste fictive** « *that establishes the mode of the ... vectors to be read* » — "" caractère, 0 numérique · résultat : **une liste** dont les composantes sont les vecteurs · nommer la liste fictive est « **more conveniently** » · la liste peut être **attachée en position 2** · **valeur unique au lieu d'une liste** → « *a single vector ... all components of which must be of the same mode as the dummy value* » · `X <- matrix(scan("light.dat", 0), ncol = 5, **byrow = TRUE**)` .

Jeux intégrés. ~100 dans le paquet `datasets`, d'autres dans les paquets recommandés · `data()` les liste · « **All the datasets supplied with R are available directly by name** » · `data(infert)` est la **convention obsolète**, encore acceptée · « *in a few cases it loads several objects* » · `data(package = "rpart")` liste, `data(Puromycin, package = "datasets")` charge · « If a package has been attached by library, its datasets are automatically included in the search »*.

Éditer. `edit(x)` ouvre un tableur — « *for making small changes once a data set has been read* » — et **assigne le résultat** : `xnew <- edit(xold)` **laisse xold intact** · `fix(xold)` « *is equivalent to xold <- edit(xold)* » · `edit(data.frame())` pour saisir de zéro.

Active Recall

« *If the file has one fewer item in its first line than in its second, this arrangement is presumed to be in force.* » (§7.1)

R **compte les items**. Si l'en-tête en a **un de moins** que la première ligne de données, la colonne supplémentaire est prise pour **une colonne d'étiquettes de ligne** — sans qu'aucun argument n'ait été donné.

C'est pourquoi `read.table("houses.data")` suffit sur le premier exemple du cours (6 noms, 7 items), tandis que le second (6 noms, 6 items) exige `header = TRUE` .

Les items s'équilibrent — autant sur la première ligne que sur les suivantes — donc **la règle de détection ne s'applique pas** et R ne présume rien. **La ligne de noms est lue comme**

une ligne de données.

Conséquence : chaque colonne contient désormais un nom de variable parmi ses valeurs, donc **toutes les colonnes deviennent des caractères** (« *non-numeric variables ... as character variables* »). Aucune erreur n'est levée à la lecture ; la panne survient plus loin, sur une opération arithmétique, avec un message sans rapport avec la lecture.

Le réflexe : après tout `read.table()`, vérifier `str()` — le mode de chaque colonne.

Par **la liste fictive** passée en deuxième argument :

```
inp <- scan("input.dat", list("", 0, 0))
```

« *The second argument is a dummy list structure that establishes the mode of the three vectors to be read. The result ... is a list whose components are the three vectors read in.* » (§7.2)

`""` annonce du **caractère**, `0` du **numérique**. On ne décrit pas le format, **on donne un exemplaire vide de ce qu'on attend**.

Et « *more conveniently, the dummy list can have named components* » : `list(id = "", x = 0, y = 0)` permet `inp$id`, `inp$x`, `inp$y`.

« *If the second argument is a single value and not a list, a single vector is read in, all components of which must be of the same mode as the dummy value.* » (§7.2)

C'est la voie du bloc homogène :

```
X <- matrix(scan("light.dat", 0), ncol = 5, byrow = TRUE)
```

`byrow = TRUE` **n'est pas décoratif**. `scan()` rend les nombres **dans l'ordre du fichier** — ligne par ligne — alors que `matrix()` remplit **en colonne-majeur** (fiche 305). Sans lui, la première colonne recevrait la première **ligne** du fichier : dimension correcte, contenu entièrement faux.

1. **Par numéro** : `label <- inp[[1]]`; `x <- inp[[2]]`; `y <- inp[[3]]` .
2. **Par nom**, si la liste fictive est nommée : `label <- inp$id`; `x <- inp$x`; `y <- inp$y` — « *more conveniently* ».
3. **En attachant la liste** : « *the list may be **attached at position 2 of the search path*** » (§7.2), ce qui rend `id` , `x` , `y` visibles directement (fiche 306, §6.3.4 — `attach()` accepte « *any object of mode "list"* »).

La deuxième est la plus robuste : ajouter une colonne ne décale pas les indices.

*« **Around 100 datasets are supplied with R (in package `datasets`)** ... To see the list of datasets currently available use `data()` . »* (§7.3)

« **All the datasets supplied with R are available directly by name. However, many packages still use the obsolete convention in which `data` was also used to load datasets** » — `data(infert)` fonctionne encore avec les paquets standard.

« *In most cases this will load **an R object of the same name**. However, **in a few cases it loads several objects**, so **see the on-line help*** ».

Pour un paquet précis (§7.3.1) : `data(package = "rpart")` **liste**, `data(Puromycin, package = "datasets")` **charge**. Et *« if a package has been attached by `library` , **its datasets are automatically included in the search** »*.

*« The command `xnew <- edit(xold)` will allow you to edit your data set `xold` , and **on completion the changed object is assigned to `xnew`** — **if you want to alter the original**

dataset `xold` , the simplest way is to use `fix(xold)` , which is equivalent to `xold <- edit(xold)` . »* (§7.4)

`edit()` **rend un objet ; il ne modifie rien** — c'est la sémantique de copie de R (fiche 302).
`fix()` **est exactement l'aller-retour** : éditer, puis réassigner au même nom.

Et `edit(data.frame())` — un data frame **vide** — sert « *to enter new data via the spreadsheet interface* ».

« *useful for making small changes* » : une correction à la souris **ne laisse aucune trace reproductible**.

« *R input facilities are simple and their requirements are fairly strict and even rather inflexible. There is a clear presumption by the designers of R that you will be able to modify your input files using other tools, such as file editors or Perl, to fit in with the requirements of R. Generally this is very simple.* » (§7)

C'est **un choix de conception explicite**, pas une lacune : R ne cherche pas à deviner un fichier mal formé, parce que deviner produit des lectures **silencieusement fausses**. Le nettoyage revient à l'outil qui sait le faire — un éditeur, `sed` ou `awk` sous UNIX (note 1), Perl.

Le corollaire pratique : devant un fichier récalcitrant, **corriger le fichier** plutôt que de chercher l'argument qui fera plier `read.table()` .

Flashcards

Question	Réponse
Comment R qualifie-t-il ses facilités d'entrée ?	« <i>simples ... strictes et même plutôt inflexibles</i> »
Que présument ses concepteurs ?	Que vous nettoierez le fichier avec d'autres outils
Deux utilitaires cités sous UNIX ?	<code>sed</code> et <code>awk</code>
Quel manuel pour aller plus loin ?	R Data Import/Export
La fonction pour lire un data frame entier ?	<code>read.table()</code>
La fonction plus primitive ?	<code>scan()</code>
Que doit contenir la première ligne du fichier ?	Un nom par variable
Et les lignes suivantes ?	Une étiquette de ligne puis les valeurs
La règle de détection des étiquettes ?	Un item de moins sur la première ligne
L'option quand il n'y a pas d'étiquettes ?	<code>header = TRUE</code>
Que déduit R de <code>header = TRUE</code> ?	Qu' aucune étiquette de ligne n'est donnée
Comment sont lus les items numériques ?	En variables numériques
Sauf lesquels ?	Les étiquettes de ligne
Comment sont lus les items non numériques ?	En variables caractères
Peut-on changer ce comportement ?	« <i>This can be changed if necessary</i> »
Quel est le 2 ^e argument de <code>scan()</code> ?	Une liste fictive
Que fixe-t-elle ?	Le mode des vecteurs à lire
Que signifie <code>""</code> dans la liste fictive ?	Du caractère
Et <code>0</code> ?	Du numérique
Que rend <code>scan()</code> avec une liste ?	Une liste de vecteurs
Comment y accéder par numéro ?	<code>inp[[1]]</code> , <code>inp[[2]]</code> ...

Question	Réponse
Comment rendre l'accès plus commode ?	Nommer la liste fictive
Troisième voie d'accès ?	Attacher la liste en position 2
Si le 2 ^e argument est une valeur unique ?	Un seul vecteur , tout du même mode
Lire une matrice de 5 colonnes ?	<code>matrix(scan(f, 0), ncol = 5, byrow = TRUE)</code>
Pourquoi <code>byrow = TRUE</code> ?	<code>scan</code> lit par ligne , <code>matrix</code> remplit par colonne
Combien de jeux fournis avec R ?	Environ 100
Dans quel paquet ?	<code>datasets</code>
Comment les lister ?	<code>data()</code>
Sont-ils disponibles par leur nom ?	Oui, directement
Que vaut alors <code>data(infert)</code> ?	La convention obsolète , encore acceptée
Combien d'objets <code>data()</code> charge-t-il ?	Un, mais parfois plusieurs
Lister les jeux d'un paquet ?	<code>data(package = "rpart")</code>
Charger un jeu précis ?	<code>data(Puromycin, package = "datasets")</code>
Après un <code>library()</code> ?	Les jeux du paquet sont automatiquement dans la recherche
Que fait <code>edit()</code> sur un data frame ?	Ouvre un environnement de type tableur
À quoi est-ce destiné ?	Des petites corrections après lecture
<code>xnew <- edit(xold)</code> modifie-t-il <code>xold</code> ?	Non
Comment modifier l'original ?	<code>fix(xold)</code>
À quoi <code>fix(x)</code> équivaut-il ?	<code>x <- edit(x)</code>
Saisir des données de zéro ?	<code>edit(data.frame())</code>
Le principal reproche à l'édition manuelle ?	Aucune trace reproductible